

Quiero que desarrolles un módulo profesional de pagos para una aplicación backend construida con **FastAPI + Python**, siguiendo principios de arquitectura limpia, seguridad, escalabilidad y separación de responsabilidades.

1. Objetivo

Construir un módulo de pagos que inicialmente permita trabajar con **Wompi en ambiente Sandbox**, pero que tenga una arquitectura preparada para agregar posteriormente:

- Mercado Pago
- ePayco
- PayU
- Stripe
- Otros proveedores

La aplicación debe permitir crear pagos, consultar su estado, recibir webhooks, validar las respuestas del proveedor y actualizar la información en PostgreSQL.

El backend debe ser completamente independiente del frontend.

2. Stack tecnológico

Utilizar:

- Python 3.12+
- FastAPI
- Pydantic v2
- SQLAlchemy 2.x
- PostgreSQL
- Alembic
- httpx
- pytest
- pytest-asyncio
- Docker
- Docker Compose
- python-dotenv / pydantic-settings

No utilizar lógica de negocio directamente dentro de los routers.

3. Arquitectura

Crear una estructura similar a:

```

app/ ├── main.py ├── core/ | ├── config.py | ├── security.py | └── database.py | ├──
api/ | └── routes/ | ├── payments.py | └── webhooks.py | ├── schemas/ | └──
payment.py | ├── models/ | └── payment.py | ├── services/ | ├── payment_service.py |
└── wompi_service.py | └── payment_factory.py | ├── repositories/ | └──
payment_repository.py | ├── integrations/ | └── wompi/ | ├── client.py | ├── schemas.py |
└── webhook.py | └── tests/ ├── test_payments.py ├── test_wompi.py └── test_webhooks.py

```

La estructura puede modificarse si existe una alternativa arquitectónica mejor, pero debe mantenerse claramente separada:

Router → Service → Repository → Database

y:

Service → Payment Provider → API externa

4. Variables de entorno

Crear un archivo `.env.example`:

PAYMENT_ENV=sandbox

DATABASE_URL=postgresql+asyncpg://postgres:postgres@db:5432/payments

WOMPI_PUBLIC_KEY= WOMPI_PRIVATE_KEY= WOMPI_EVENTS_SECRET=

WOMPI_API_URL=https://sandbox.wompi.co/v1

APP_URL=http://localhost:8000

LOG_LEVEL=INFO

Nunca colocar credenciales reales dentro del código fuente.

Agregar `.env` al `.gitignore`.

5. Modelo de pagos

Crear una tabla PostgreSQL `payments`.

Debe contener como mínimo:

- id
- reference
- provider
- provider_transaction_id
- amount
- currency
- status
- payment_method
- customer_email
- customer_name
- redirect_url
- checkout_url
- raw_response
- webhook_payload
- created_at
- updated_at

Utilizar UUID como identificador interno.

El campo `reference` debe ser único.

Crear índices para:

- reference
- provider_transaction_id
- status
- created_at

6. Estados de pago

Definir un Enum:

PENDING APPROVED DECLINED VOIDED ERROR EXPIRED

No permitir que cualquier parte de la aplicación modifique arbitrariamente los estados.

Implementar una función centralizada para controlar las transiciones válidas.

Por ejemplo:

PENDING → APPROVED PENDING → DECLINED PENDING → ERROR PENDING → EXPIRED

Evitar que un pago APPROVED vuelva accidentalmente a PENDING.

7. Endpoint para crear pago

Crear:

POST /api/v1/payments

Request:

```
{ "amount": 50000, "currency": "COP", "customer_email": "cliente@example.com", "customer_name": "Cliente Demo", "description": "Pago prueba", "provider": "wompi" }
```

El sistema debe:

1. Validar los datos.
2. Generar una referencia única.
3. Crear el registro PENDING.
4. Comunicarse con Wompi Sandbox.
5. Procesar la respuesta.
6. Guardar el ID de transacción.
7. Guardar el checkout/payment URL si aplica.
8. Retornar una respuesta limpia al frontend.

No retornar secretos ni información sensible.

Respuesta:

```
{ "payment_id": "UUID", "reference": "IAZY-20260905-XXXX", "status": "PENDING", "checkout_url": "...", "provider": "wompi" }
```

8. Consultar pago

Crear:

GET /api/v1/payments/{payment_id}

Debe devolver:

- id
- reference
- amount
- currency
- provider
- provider_transaction_id
- status

- payment_method
- created_at
- updated_at

No devolver las API keys.

9. Consultar por referencia

Crear:

GET /api/v1/payments/reference/{reference}

Debe buscar el pago en PostgreSQL y devolver su estado actual.

10. Consultar estado directamente al proveedor

Crear:

POST /api/v1/payments/{payment_id}/sync

Este endpoint debe:

1. Obtener el pago desde PostgreSQL.
 2. Obtener el `provider_transaction_id`.
 3. Consultar Wompi.
 4. Comparar el estado recibido.
 5. Actualizar PostgreSQL.
 6. Registrar la respuesta.
 7. Retornar el estado actualizado.
-

11. Webhook

Crear:

POST /api/v1/webhooks/wompi

El webhook debe:

1. Recibir el evento.
2. Validar la estructura.
3. Validar la firma/hash de seguridad según la documentación oficial de Wompi.
4. Obtener la referencia.

5. Buscar el pago.
6. Verificar que corresponda al pago esperado.
7. Actualizar el estado.
8. Guardar el payload recibido.
9. Responder rápidamente HTTP 200.

El webhook debe ser **idempotente**.

Si Wompi envía el mismo evento varias veces, no debe generar pagos duplicados ni alterar incorrectamente el estado.

12. Idempotencia

Implementar idempotencia para la creación de pagos.

Aceptar opcionalmente:

Idempotency-Key

Si llega una solicitud repetida con la misma llave, devolver el pago previamente creado en lugar de crear otro.

También utilizar restricciones únicas en PostgreSQL donde corresponda.

13. Abstracción de proveedores

Crear una interfaz:

PaymentProvider

Con métodos como:

create_payment() get_payment() refund_payment() validate_webhook()

Crear:

WompiProvider(PaymentProvider)

Posteriormente debe ser posible implementar:

MercadoPagoProvider EPaycoProvider PayUProvider StripeProvider

sin modificar significativamente `PaymentService`.

Ejemplo conceptual:



Utilizar Factory o Strategy Pattern para seleccionar el proveedor.

14. Cliente HTTP

Utilizar `httpx.AsyncClient`.

Implementar:

- timeout
- manejo de errores HTTP
- logging
- retries controlados para errores transitorios
- validación de respuestas
- cierre correcto del cliente

No utilizar requests si la operación puede ser asíncrona.

15. Manejo de errores

Crear excepciones personalizadas:

`PaymentException` `PaymentProviderException` `PaymentNotFoundException` `PaymentValidationException`
`WebhookValidationException`

Utilizar exception handlers de FastAPI para devolver respuestas consistentes.

Formato:

```
{ "success": false, "error": { "code": "PAYMENT_PROVIDER_ERROR", "message": "No fue posible procesar el pago" } }
```

No exponer stack traces ni secretos en producción.

16. Seguridad

Implementar:

- Validación estricta con Pydantic.
- Variables de entorno para secretos.
- HTTPS preparado para producción.
- Validación de webhooks.
- Logs sin información sensible.
- No almacenar CVV.
- No almacenar números completos de tarjetas.
- No guardar tokens secretos en PostgreSQL.
- No imprimir API keys.
- No incluir secretos en respuestas.
- CORS configurable mediante variables de entorno.

La aplicación no debe manejar directamente datos sensibles de tarjeta si Wompi proporciona mecanismos de tokenización/checkout que permitan evitarlo.

17. Logging

Implementar logging estructurado.

Registrar:

- payment_id
- reference
- provider
- transaction_id
- endpoint
- duración
- resultado
- errores

Nunca registrar:

- API keys
 - passwords
 - CVV
 - tokens privados
 - datos completos de tarjetas
-

18. Tests

Crear tests automatizados con pytest.

Probar como mínimo:

Crear pago

- pago válido
- monto inválido
- moneda inválida
- email inválido
- proveedor inexistente
- error de Wompi

Consultar pago

- pago existente
- pago inexistente

Webhook

- webhook válido
- firma inválida
- pago inexistente
- evento duplicado
- transición de estado inválida

Idempotencia

Enviar dos veces la misma solicitud y comprobar que solamente existe un pago.

Utilizar mocks para las llamadas externas durante los tests.

No hacer llamadas reales a Wompi durante los tests unitarios.

19. Docker

Crear:

Dockerfile

docker-compose.yml

Servicios:

- api
- postgres

Opcionalmente preparar Redis para futuras tareas asíncronas.

La aplicación debe poder levantarse con:

`docker compose up --build`

y quedar disponible en:

`http://localhost:8000`

Swagger:

`http://localhost:8000/docs`

ReDoc:

`http://localhost:8000/redoc`

20. Migraciones

Configurar Alembic.

Crear la migración inicial para `payments`.

Documentar:

`alembic upgrade head`

y:

`alembic downgrade -1`

21. Documentación

FastAPI debe generar automáticamente OpenAPI.

Agregar:

- descripción de endpoints
- request examples
- response examples
- códigos HTTP
- errores
- autenticación si aplica

Crear también un `README.md` explicando:

1. Instalación.
2. Variables de entorno.
3. Configuración de PostgreSQL.
4. Ejecución local.
5. Docker.
6. Sandbox de Wompi.
7. Cómo obtener las credenciales.
8. Cómo probar desde Swagger.
9. Cómo probar con Postman.
10. Cómo probar webhooks.
11. Cómo ejecutar pytest.
12. Cómo pasar de Sandbox a Producción.

22. Postman

Crear una colección de Postman o un archivo JSON importable con:

POST /api/v1/payments

GET /api/v1/payments/{payment_id}

GET /api/v1/payments/reference/{reference}

POST /api/v1/payments/{payment_id}/sync

POST /api/v1/webhooks/wompi

Incluir variables:

base_url payment_id reference

23. Health check

Crear:

GET /health

Respuesta:

```
{ "status": "ok" }
```

Crear también:

GET /health/db

que compruebe la conexión con PostgreSQL.

24. Calidad del código

El código debe:

- utilizar type hints
- utilizar async/await correctamente
- seguir PEP8
- tener funciones pequeñas
- tener separación de responsabilidades
- evitar código duplicado
- utilizar Dependency Injection de FastAPI cuando sea apropiado
- tener comentarios únicamente donde aporten valor
- ser mantenible y preparado para producción

No generar código ficticio para endpoints de Wompi.

Antes de implementar la integración específica, revisar la documentación oficial y utilizar exactamente los endpoints, headers, autenticación, formatos de request, formatos de response y mecanismos de firma/webhook vigentes para **Wompi Colombia Sandbox**.

Si alguna parte de la documentación oficial contradice este prompt, prevalece la documentación oficial actual de Wompi.

25. Entregables

Quiero que generes:

1. Estructura completa del proyecto.
2. Código de todos los archivos necesarios.
3. `.env.example`.
4. `.gitignore`.
5. `Dockerfile`.
6. `docker-compose.yml`.
7. Configuración de Alembic.
8. Modelo SQLAlchemy.
9. Schemas Pydantic.
10. Repository.
11. Payment Service.
12. Interface PaymentProvider.
13. WompiProvider.
14. Wompi HTTP client.
15. Webhook handler.
16. Routers.
17. Manejo global de excepciones.
18. Tests con pytest.
19. Colección Postman.
20. README.
21. Instrucciones exactas para ejecutar el proyecto.

IMPORTANTE:

No quiero solamente ejemplos parciales.

Quiero una implementación funcional y coherente de extremo a extremo:

Swagger/Postman ↓ FastAPI ↓ PaymentService ↓ WompiProvider ↓ Wompi Sandbox ↓ Webhook ↓
FastAPI ↓ PostgreSQL ↓ Estado final del pago

Primero analiza la arquitectura completa y después genera el código necesario archivo por archivo.

No inventes endpoints ni parámetros de Wompi. Utiliza la documentación oficial vigente.